# NINE!

## Documentación

**Proyecto de:**

**Programación**

**Alumnos: Ulises SIlvera, Facundo Noble y Renato Viera**

**Profesor: Sebastian de los Angeles**

**Fecha:**

## 1. Introducción

**Nine!** es un juego de cartas basado en el UNO, desarrollado en **Java** con interfaz gráfica **Swing**. Cada jugador empieza con 9 cartas y gana el primero que se queda sin ninguna. Además de las cartas clásicas (números, Bloqueo, Reversa, +2, Comodín y +4), Nine! suma cartas propias: **Menos Dos**, **Espiar Mazo**, **Espiar Baraja** y **Cambio de Mano**, y un sistema de **habilidades** que cada jugador elige al empezar.

El juego se puede jugar de dos formas: **local** (2 jugadores en una misma computadora) y **en red** (2 a 4 jugadores, cada uno en su computadora, conectados por sockets). Los resultados (victorias, derrotas y partidas jugadas) se guardan en una base de datos **MariaDB** llamada JuegoNINE.

### Reglas principales

| Regla | Cómo funciona |
|---|---|
| Objetivo | 9 cartas cada uno; gana quien se queda sin cartas. |
| Jugar | Una sola carta por turno: del color de la mesa, del mismo número o tipo. Las cartas especiales siempre se pueden tirar. |
| Color | Solo el Comodín y el +4 cambian el color de la mesa. |
| Robar | Solo si no tienes ninguna carta jugable; de a una y hasta 3 por turno. Si la carta sirve, la tiras; si llegas a 3 sin poder jugar, pierdes el turno. |
| +2 / +4 | El rival roba 2 o 4 cartas y juega. |
| Bloqueo / Reversa | Pierde el turno el siguiente (con 2 jugadores, vuelve a jugar quien la tiró). |
| Habilidades | Cada jugador elige 1 de 2 sorteadas: Dupeo, Espejo, Protección o Intercambio (2 usos). |
| Tiempo | 30 segundos por turno; el reloj se pone rojo con 15 s o menos. |
| Rendirse | Con 2 jugadores gana el otro; con 3 o 4 quedas eliminado y sigue la partida. |
| Mazo | Nunca se agota: se rebaraja el descarte y, si hace falta, se agrega un mazonuevo. |

## 2. Ideas principales

El diseño del juego se apoya en cinco ideas:

### Separar el juego en capas

El código está en paquetes con una responsabilidad cada uno: `modelo` (cartas, mazo, jugadores), `lógica` (reglas), `interfaz` (ventanas), `red` (servidor y cliente) y `BD` (base de datos). Las reglas estan en una sola clase, `JuegoNineControlador`, que no sabe nada de ventanas. Así el modo local y el modo en red usan **exactamente las mismas reglas**.

### El servidor manda en el modo en red

Los clientes solo envían lo que quieren hacer (jugar, robar, rendirse) y dibujan lo que el servidor les responde. El servidor valida cada jugada y a cada jugador le manda **solo su propia mano**. Los mensajes son líneas de texto con campos separados por `|` (por ejemplo `JUGAR|ROJO,NUMERO,5|-`).

### Hilos para que nada se congele

Cada jugador conectado tiene un hilo lector y un hilo escritor con una cola de mensajes; el servidor procesa un mensaje a la vez con `synchronized`. El reloj de turno y el guardado en la base de datos también van en hilos aparte.

### Cartas especiales (NINE) y habilidades

Espiar muestra información oculta, Cambio de Mano intercambia manos, Menos Dos descarta cartas y las habilidades permiten duplicar una carta, devolver un ataque, bloquearlo o intercambiar cartas con el rival. Cada jugador elige entre **dos habilidades sorteadas**, no entre todas.

### Estadísticas

Al terminar la partida se guarda quién ganó y quiénes perdieron (incluidos los que se rindieron) con `PreparedStatement` y `INSERT ... ON DUPLICATE KEY UPDATE`. Si la base está apagada, la partida sigue normal.

**Estructura del proyecto**

| Paquete | Clases |
|---|---|
| `modelo` | Color, TipoCarta, TipoHabilidad, Habilidad, Carta, Jugador, Mazo |
| `logica` | JuegoNineControlador |
| `interfaz` | MenuPrincipal, VentanaJuego (local), VentanaCliente (red), BotonCarta, GestorImagenes, DialogoHabilidad |
| `red` | ServidorNine, ManejadorCliente, ClienteNine, Protocolo |
| `BD` | GestorBD |
| `main` | JuegoPetNine (método main) |

## 3. Fundamentación

Por qué se tomaron estas decisiones:

- **Java y Swing:** son las herramientas vistas en la materia, funcionan en cualquier computadora y permiten una interfaz de escritorio completa sin librerías externas.
- **Programación orientada a objetos:** cada concepto del juego es una clase (Carta, Mazo, Jugador). Los `enum` evitan errores de escritura al representar colores, tipos de carta y habilidades.
- **Controlador único de reglas:** si las reglas estuvieran dentro de cada ventana, habría que corregir cada error dos veces (local y red) y podrían divergir. Con un único controlador se corrige una sola vez.
- **Sockets TCP con protocolo de texto:** TCP garantiza que los mensajes lleguen completos y en orden. Un protocolo de texto es fácil de leer y de depurar (se puede ver en consola qué se envía).
- **Cola de mensajes por jugador:** si un jugador tiene conexión lenta, solo se retrasa él; el servidor no se frena ni espera por él.
- **Base de datos (MariaDB) con JDBC:** las estadísticas son datos estructurados (jugadores y sus partidas), y SQL permite consultarlos y ordenarlos después, por ejemplo para un ranking. `PreparedStatement` evita la inyección SQL.
- **Reglas del UNO original:** una carta por turno y robar acotado hacen el juego más ágil y fácil de entender. Las ideas propias (cartas Nine!, habilidades) se agregan sin romper esa base.

## 4. Bugs o errores encontrados y cómo se solucionaron

### Las estadísticas no se guardaban en red

**Qué pasaba:** En modo local se guardaban victorias y derrotas; en red no.

**Causa:** Solo la ventana local llamaba a `GestorBD`; el servidor nunca lo hacía.

**Solución:** El servidor guarda el resultado al terminar la partida, en un hilo aparte para no frenar el juego.

```java
GestorBD.registrarVictoria(ganador);
  for (String p : perdedores) GestorBD.registrarDerrota(p);
```

### Las cartas especiales del Nine! cambiaban el color de la mesa

**Qué pasaba:** Espiar Mazo, Espiar Baraja, Menos Dos y Cambio de Mano pedían

elegir color.

**Causa:** Se comprobaba `getColor() == Color.ESPECIAL`, que incluye a

todas las especiales.

**Solución:** Nuevo método que solo acepta Comodín y +4.

```java
  public boolean cambiaColor() { return tipo == TipoCarta.COMODIN
  || tipo == TipoCarta.SUMA_CUATRO; }
```

### Espiar mostraba información duplicada o equivocada (local)

**Qué pasaba:** Al espiar, el efecto se ejecutaba dos veces o con datos incorrectos.

**Causa:** Los botones de la mano se recreaban agregando otro `ActionListener`

sin quitar el anterior.

**Solución:** Se limpia el panel y se crea un solo listener por botón al redibujar.

### El mazo se agotaba

**Qué pasaba:** Con muchas cartas robadas, `robar()` lanzaba `EmptyStackException`.

**Causa:** Nadie rebarajaba el descarte.

**Solución:** Se asegura que siempre haya cartas: primero se rebaraja el descarte y, si no alcanza, se agrega un mazo nuevo.

```java
private void asegurarCartas() { if (cartasRobo.isEmpty()) reorganizarMazo(); if (cartasRobo.isEmpty()) { generarCartas(); Collections.shuffle(cartasRobo); } }
```

### Robar hacía perder el turno y no tenía reglas

**Qué pasaba:** Robar pasaba el turno; después se podía robar sin límite, incluso con cartas jugables.

**Causa:** No estaba definido cuándo y cuánto se puede robar.

**Solución:** El controlador devuelve un resultado (`TIENE_JUGABLE`, `JUGABLE`, `SIGUE`, `PIERDE_TURNO`): solo se roba sin cartas jugables, de a una y hasta 3 por turno.

### El turno volvía a quien no debía (2 jugadores)

**Qué pasaba:** Tras +2, +4 o Bloqueo, el turno regresaba al jugador que acababa de jugar de forma inesperada.

**Causa:** Los saltos se acumulaban sin límite y con 2 jugadores devolvían el turno.

**Solución:** +2 y +4 solo hacen robar al rival (que juega). Los saltos se limitan para que con 3 o más jugadores nunca vuelvan a quien jugó.

```java
saltosPendientes = Math.min(saltosPendientes, jugadores.size() == 2 ? 1 : jugadores.size() - 2);
```

### Partida trabada con Cambio de Mano

**Qué pasaba:** Si tirabas tu última carta con Cambio de Mano, la partida no terminaba.

**Causa:** El rival recibía la mano vacía y solo se revisaba al jugador que jugó.

**Solución:** Se revisa a todos los jugadores, en local y en el servidor.

### Cancelar un cuadro hacía perder cartas o usos

**Qué pasaba:** Cancelar en Menos Dos, Comodín, Intercambio o Protección dejaba la jugada a medias (carta perdida, uso gastado o color rojo por defecto).

**Causa:** Se modificaba la mano antes de terminar de elegir.

**Solución:** Se elige todo primero y solo entonces se cambia el estado. Protección pasó a ser pasiva, como en red.

### Habilidad sin elegir en local

**Qué pasaba:** En local el jugador no podía elegir su habilidad, y en red se podía

elegir entre todas.

**Causa:** Cada modo resolvía la elección por su cuenta.

**Solución:** Un cuadro único (`DialogoHabilidad`) ofrece dos habilidades sorteadas distintas, en local y en red; el servidor valida que la elegida sea una de las dos.

### El turno no era claro y el tiempo no existía

**Qué pasaba:** Era difícil saber de quién era el turno, y un jugador inactivo

trababa la partida.

**Causa:** Faltaba un indicador y un límite de tiempo.

**Solución:** Cartel «¡Es el turno de ...!», reloj de 30 s (rojo con 15 s o menos) que pasa el turno solo; en red decide el servidor y en local el reloj se pausa mientras hay un cuadro abierto.

## 5. Ideas que faltaron

Ideas que al final no entraron al juego final:

- **Pantalla de ranking:** los datos de victorias y derrotas ya están guardados en la base; falta mostrarlos en una ventana.
- **Habilidad definitiva**: Una habilidad **muy buena** que se pueda tirar una vez sola por partida.
- S**onidos y animaciones** al tirar cartas para mejorar la experiencia.
- **Agregar un login:** crear una interfaz para que los usuarios puedan tener sus usuarios y contraseñas.